Search in course

Search

English

B

1. CS 498: Cloud Computing Applications
2. Week 16
3. Machine Problem 2 (V2) : Load Balancing + Auto Scaling

PreviousNext

Programming Assignment: Machine Problem 2 (V2) : Load Balancing + Auto Scaling

You have not submitted.

Deadline

Pass this assignment by May 11, 11:59 PM CDT

InstructionsMy submissionsDiscussions

1. Overview

This MP has two sections. One is about AWS Elastic Load Balancing (ELB), and the other is about Amazon EC2 Auto Scaling.

AWS Elastic Load Balancing (ELB) spreads user traffic across many instances of your applications. A load balancer decreases the possibility of performance issues in your applications by distributing the load. It also helps with the reliability and scalability of your product, which may be running on several virtual machines. AWS EC2 enables you to create virtual machines on AWS infrastructure that run various operating systems, including multiple flavors of Linux, Windows Server, and macOS. In the first section of this MP, we will launch a load balancer to balance requests between 2 web servers hosted on Amazon EC2.

Amazon EC2 Auto Scaling is an AWS feature that automatically optimizes the number of EC2 instances available to your workloads or app based on workloads' performance and pre-defined conditions. In the second section of this MP, we'll create an AWS auto scaling group to monitor the CPU usage of the EC2 instances and decide when to scale up the number of instances.

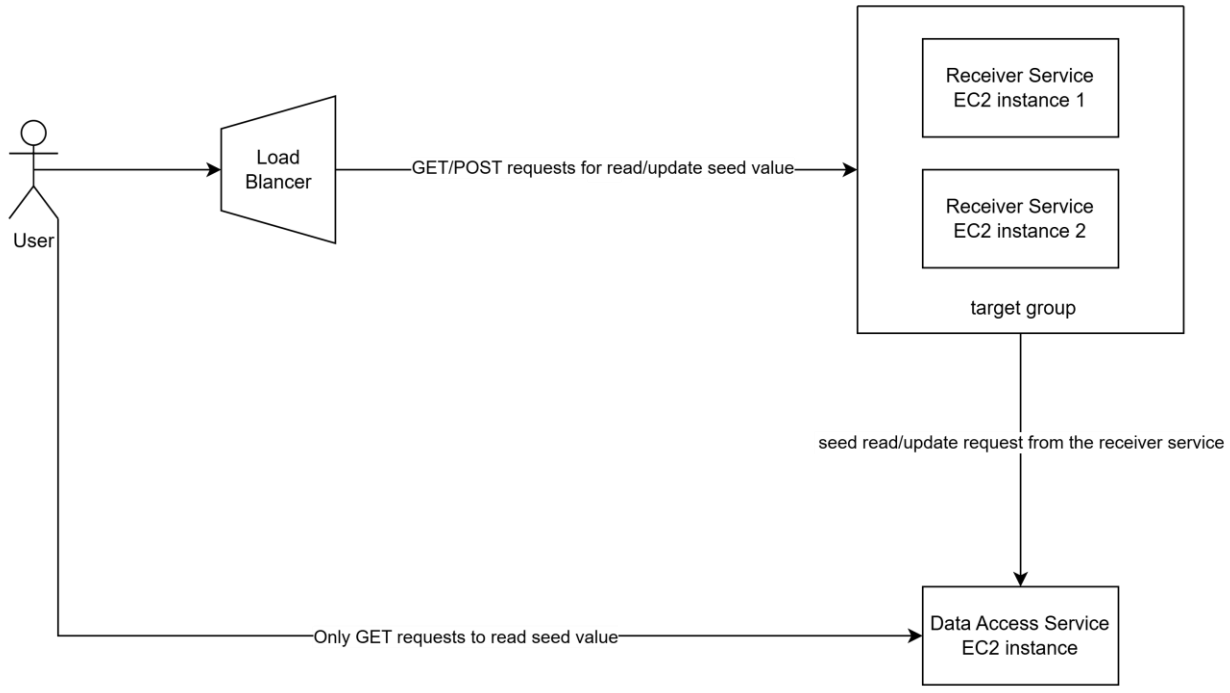
Note: To solve this problem you may want to familiarize yourself with IAM, which are covered in week 3. read . Note: If you receive an error in grading output by autograder for your submission , please follow the FAQ section of this document, and campuswire posts for suggestions to resolve the errror.

2. Requirements

You need to use the terminal or command prompt on your computer to ssh into the AWS instances. For Mac and Linux users, ssh is already available in the terminal. For Windows users, you can install and use PuTTY.

3. Procedure

Section 1. AWS Load Balancer



In this section, you will develop a two tier architecture of REST services as mentioned in the above diagram. The Receiver application would receiver user requests to retrieve and update an integer seed value from the Data Access service which holds the seed value and updates it based on the user requests

Step 1.1. Setup for Data Access Service

A. Launch an EC2 Instance

1. We suggest using the latest Amazon Linux AMI as the operating system and the smallest EC2 instance possible (e.g., Nano or Micro) to make the cost of your instances as small as possible (or free). Refer to the link below for the specific steps.**https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EC2_GetStarted.html**
2. Ensure you can connect to the 2 EC2 instances using the terminal (or the embedded terminal in the EC2 console on your browser) via ssh.
3. **Record the subnet ID** on which this EC2 instance is created. This will be used to create NACL in the later steps.

B. Develop Data Access Service

Create a web server application using the provided template on this EC2 instance with 2 APIs:

1. **POST –** Update Seed Value
    - **Port:** 8080
    - **Functionality:** It accepts an HTTP POST request to "/" with a JSON body, e.g., {"num": 100}. This request updates the stored seed value (default - 0).
2. **GET –** Retrieve Seed Value
    - **Port:** 5000
    - **Functionality:** Accepts an HTTP GET request to "/" and returns the current seed value as a string (e.g., "100").

data_access_service_template

PY File

Step 1.2. Setup for Receiver Service

A. Launch Receiver EC2 Instances

1. **Create Two Identical Instances:**

- Use the latest Amazon Linux AMI and a small instance type.

2. **Subnet Details:**
    - Launch these instances in a **subnet different** from the one used for the Data Access Service.
    - Both Receiver Service instances should be assigned the **same subnet,** which must be different from the Data Access Service subnet.
3. **Verify SSH Connectivity:**
    - Connect each instance using SSH (or the EC2 console embedded terminal).
4. **Reference:** Follow the steps in the AWS EC2 Getting Started Guide.

B. Develop the Receiver Application

On each EC2 instance, build a simple web server application using the provided template that will interact with the Data Access Service:

1. **Port:** 8080
2. **Functionality:**
    - **HTTP POST "/" Request:**
        - Accepts a JSON body (e.g., {"num": 100}).
        - Call Data Access Service to update the seed value.
    - **HTTP GET "/" Request:**
        - Call Data Access Service to retrieve and return the current seed value as a string.

receiver_service_template

PY File

You can use your preferred programming languages or development environments to implement the applications in Step 1.1 and Step 1.2. However, we will only assist you with Python using Flask. You need to install the required dependencies (install pip and then Flask "*pip3 install flask"* along with its required dependencies if you are using Python) in

both EC2 instances and then start the server. Refer to the sections on Environment and HTTP Methods in Tutorialspoint's Flask tutorial for more details.

Step1.3. NACL Configuration

1. **Subnet Assignments:**
    - **Subnet1:** Data Access Service subnet.
    - **Subnet2:** Receiver Service subnet.
2. **Create and Configure the NACL:**
    - Create a Network Access Control List (NACL) for Subnet1.
    - Configure NACL rules to allow inbound traffic on port 8080 from Subnet2 only and allow all traffic on port 5000.
3. **Reference:** You can find documentation on setting up NACLs here: https://docs.aws.amazon.com/vpc/latest/userguide/create-network-acl.htmlhttps://docs.aws.amazon.com/vpc/latest/userguide/nacl-examples.html

Step 1.4 Security Group Setup

- Configure a security group with inbound rules to allow traffic on ports **5000** (read traffic) and **8080** (update traffic).

Step 1.5 Validation:

You can validate the NACL and security group setup by the following steps:-

1. You should be able to update the seed value and retrieve the seed value using the load balancer URL.

2.You should be restricted from updating the seed value using the public IP of Data Access Service, but you should be able to retrieve the seed value using the public IP of Data Access Service.

Step 1.6: Load balancing web servers

You will launch an AWS Application Load Balancer, which will allow you to distribute HTTP requests between the two instances of the Receiver Service you created. To get started, you might want to look at the following document.
https://docs.aws.amazon.com/elasticloadbalancing/latest/application/application-load-balancer-getting-started.html

Connect your load balancer to a target group with the active EC2 instances of the Receiver service. Make sure to map the web server port to the load balancer correctly.

When configuring the load balancer, ensure you do not use the default security group. Enable the HTTP protocol and allow traffic to be routed from anywhere when creating a custom security group. You will not pass this assignment unless our autograder can connect to your system via the internet.

To make the server code operate in the load balancer, you must edit the target group's health check ping path to "/".

To check if your load balancer is running correctly, first make sure that the two EC2 instances are healthy in the target group and then go to the DNS address listed in the AWS load balancer's description and look for the seed value.

If you face the problem of ***unhealthy instances***, please follow the suggestions mentioned in the FAQ section (Question 7 - Port Mapping FAQ) at the end of this document.

If you encounter any other issues, use Google Search, AWS documentation, or Campuswire.

Step 1.7 (Optional) EC2 Cloudwatch Debugging Logs

Amazon CloudWatch is a service for monitoring and observing data. You can debug your web application by creating a log group and monitoring the logs. The Python sample code for creating two custom logs and forwarding them to Cloudwatch is provided below. You can insert the sample code in your lambda function. Refer to the blog for more examples.

```
1
2
3
4
5
6
7
8
9
10
```

```
37
38
39
40
import boto3
import time
from datetime import datetime

AWS_REGION = "us-east-1"

client = boto3.client('logs', region_name=AWS_REGION)

message = "YOUR_MESSAGE" # replace here with your custom logging message
logGroupName = 'YOUR_GROUPNAME' # replace here with your log group name
logStreamName = 'YOUR_STREAMLOG_NAME' # replace here with your stream log name

# initial Cloudwatch configuration
response = client.create_log_stream(
   logGroupName=logGroupName,
   logStreamName=logStreamName
)

# log1
seq_token = None
log_event = {
   'logGroupName': logGroupName,
```

```
  'logStreamName': logStreamName,
  'logEvents': [
    {
      'timestamp': int(round(time.time() * 1000)),
      'message': message
    },
  ],
}
log_event['sequenceToken'] = seq_token
log1_response = client.put_log_events(**log_event)

# sleep for one second
time.sleep(1)

# log2
seq_token = log1_response['nextSequenceToken']
log_event = {
  'logGroupName': logGroupName,
```

Section 2. Auto Scaling

In this section, you'll use Amazon Auto Scaling to dynamically control the number of EC2 instances based on the CPU usage.

Step 2.1. Implement a HTTP server

- Like in Section 1, create an EC2 instance for server program development. We suggest using the Amazon Linux AMI or Ubuntu as the operating system.
- Use the **t2.micro** (free tier) instance.

- **Notice that this EC2 instance is only for developing and testing your HTTP server program, so that you can create your launch template later. It will not be used for the auto scaling.**
- Ensure you can connect to the EC2 instance using terminal (or the embedded terminal in the EC2 console on your browser) via ssh.
- You will write a simple web server application called **serve.py** which is similar to what you wrote in Section 1 (we still recommend using Python + Flask). This program will then run on the EC2 server. The application handles the following two HTTP REST requests:
    - HTTP POST "/"
        - Once receive the POST request, your server program should create a separate process for running "stress_cpu.py" (**attached below**). "stress_cpu.py" runs an intensive computation loop to stress the CPUs to 100% utilization. This script is for triggering the auto scaling policy that you will setup later.
        - In Python, use **subprocess.Popen()** to create a separate process so that your HTTP server is non-blocking. That is to say, after receiving a POST request for stressing the CPU on your EC2 instance, your server program should still be able to handle further POST/GET requests.
    - HTTP GET "/"
        - Your server program should return the private IP address of the EC2 instance. In Python, you can **import socket** and use **socket.gethostname()** to get the IP address.

stress_cpu

PY File

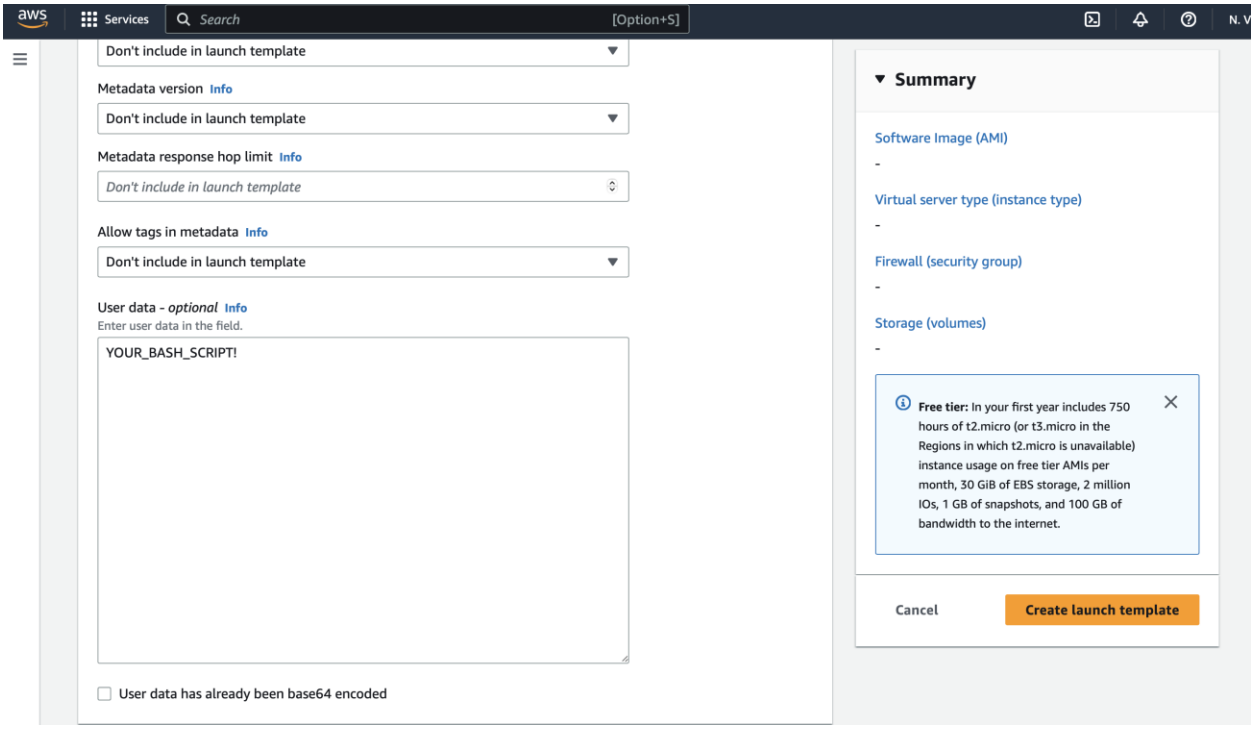
- Run your server program on a the same port as you did in Section 1, so that you can reuse the security group(with the same inbound rules) created earlier in Section 1.
- After finishing the development of your HTTP server, you need to upload your code to a Github repo (**make sure it's a private repo**). You will need to create a personal access token on Github which will be used in the bash script in Step 2.2. The code

will be used in the following steps. And the EC2 instance for developing and testing your server program can be terminated now.

- For security reasons, when creating your **GitHub Personal Access Token (PAT)**, ensure that it **only has access to the MP2 repository** instead of granting access to all repositories in your account.

Step 2.2. Create a Launch Template

- A launch template defines the configurations of launching an EC2 instance. When the auto scaling policy is triggered, Amazon EC2 Auto Scaling can automatically creates a new EC2 instances based on a launch template. Follow "Step1: Create a launch Template" in this tutorial to create a launch template: https://docs.aws.amazon.com/autoscaling/ec2/userguide/get-started-with-ec2-auto-scaling.html
- When creating your launch template, under **Advanced details -> User data**, you need to put a Bash script to automatically install all your dependencies and run your server code.



- A Bash script template looks like the following:

For Amazon Linux Server

2

3

4

5

6

7

8

9

10

11

12

```
#!/bin/bash
export HOME=/home/ec2-user
sudo yum update
sudo yum install stress-ng -y
sudo yum install htop -y
sudo yum install python3-pip -y
pip3 install flask
sudo yum install git -y
cd /home/ec2-user
git clone YOUR_CODECOMMIT_REPO_LINK (https://<github-personal-access-token>@github.com/username/repository_name)
cd /home/ec2-user/YOUR_CODECOMMIT_REPO_NAME
python3 serve.py
```

For Ubuntu Server (use **Ubuntu Server 22.04 LTS (HVM)** as AMI**)**

1

2

3

4

5

6

7

8

9

10

11

12

```
#!/bin/bash
export HOME=/home/ubuntu
sudo apt-get update
sudo apt-get install stress-ng -y
sudo apt-get install htop -y
sudo apt-get install python3-pip -y
pip3 install flask
sudo apt-get install git -y
cd /home/ubuntu
git clone YOUR_CODECOMMIT_REPO_LINK (https://<personal-access-token>@github.com/username/repository_name)
cd /home/ec2-user/YOUR_CODECOMMIT_REPO_NAME
python3 serve.py
```

- To test this step, create an EC2 instance based on your launch template, and see if it can automatically download all the dependencies and run your serve.py .

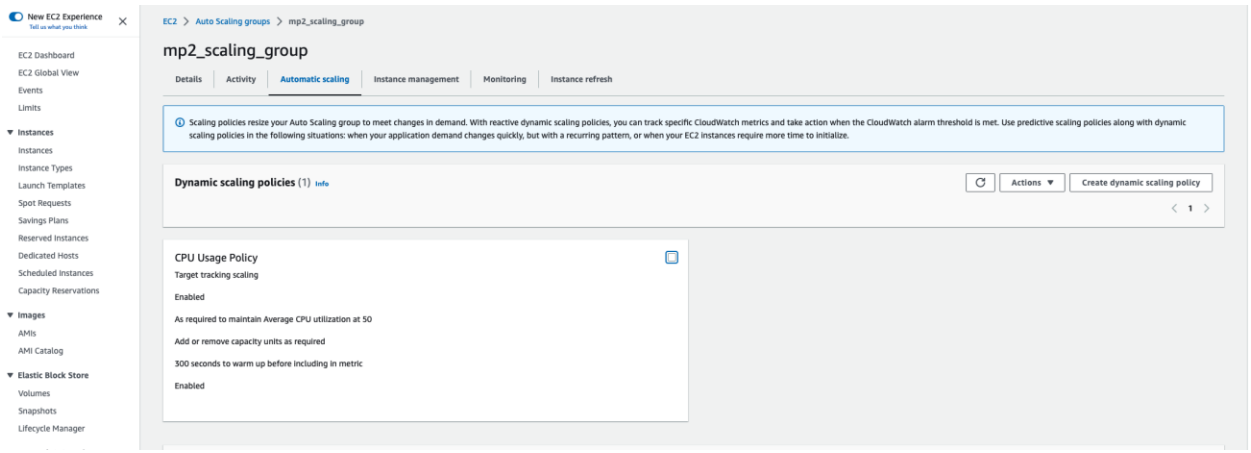
Step 2.3. Create auto scaling group

- Follow "Step 2: Create a single-instance auto scaling group" in this tutorial to create your auto scaling group:https://docs.aws.amazon.com/autoscaling/ec2/userguide/get-started-with-ec2-auto-scaling.html
- When creating auto scaling group, in **Configure advanced options - *optional*** page, create a new load balancer**.** This allows the EC2 instances automatically created by the auto scaling group to be under the management of your load balancer. Select the load balancer scheme as **"internet-facing"**.

Step 2.4. Set up auto scaling policy

- Following the section "**Create a target tracking scaling policy (console)**" in this tutorial to learn how to set up your target tracking scaling policy:https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html
- Set up your auto scaling group size by:
    - Minimum instance number = 0, maximum instance number = 3, desired instance number =1
- You'll use the **target tracking scaling policy** to monitor the CPU usage on your EC2 instances. Following is the policy that you'll set up:

1. When the CPU usage is above 50%, your auto scaling group should automatically increase the number of instance by one
2. When the CPU usage is below 50%, your auto scaling group should automatically decrease the number of instance by one
3. Set the cooldown time to be 300 seconds, in case the policy is triggered too many times



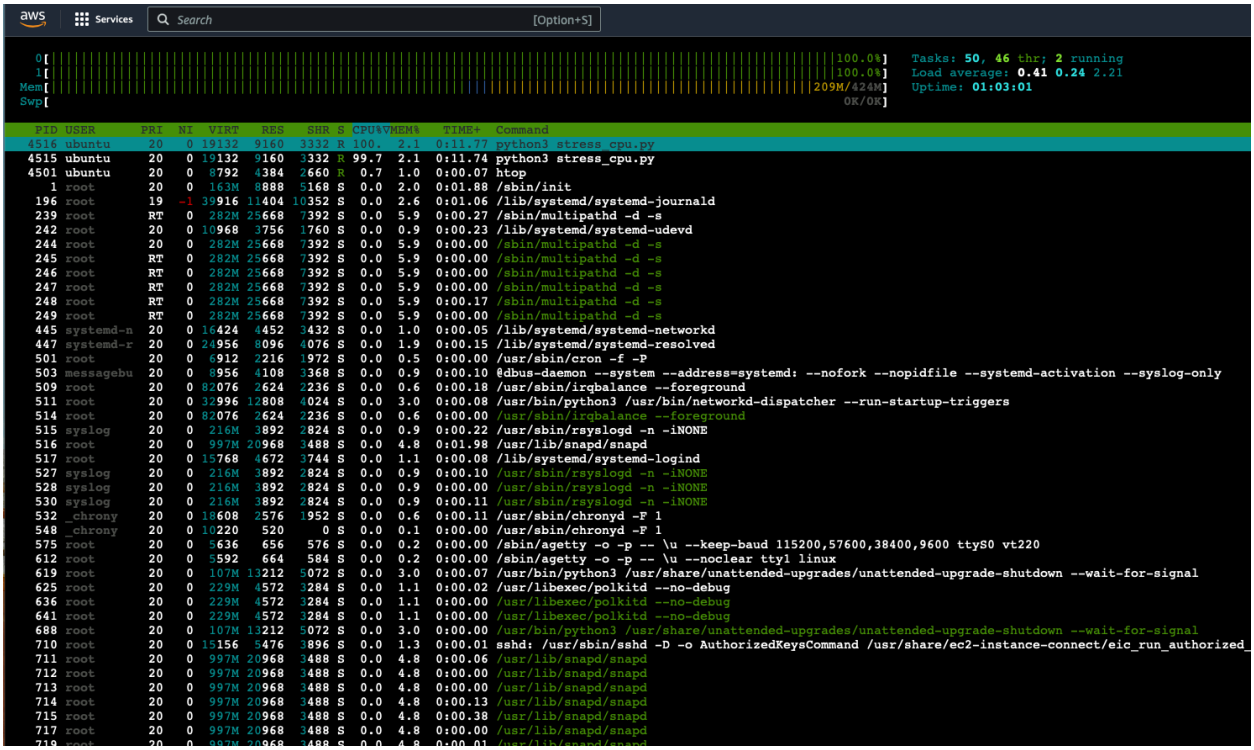
By this point, you should have a functional auto scaling group monitoring on the CPU usage of your EC2 instances, a load balancer for rerouting the HTTP requests to EC2 instances. Your auto scaling group can create EC2 instances based on your launch template. When an EC2 instance is created, it automatically download the dependencies and run your HTTP server.

Step 2.5. Test your auto scaling group

Before submitting your auto scaling group to the autograder, follow these steps to test your auto scaling group:

1. Make sure your auto scaling group has 1 active EC2 instance, and the maximum instance number is set to 3.

2. Send 1 POST request to the load balancer bound to your auto scaling group. Check if the EC2 instance can successfully receive the request and run "stress_cpu.py". Check it by using "htop" command in terminal on your EC2 instance to show the current CPU usage. If "stress_cpu.py" is running, the CPU usage should stay 100% for around 1 minute.



3. After the first "stress_cpu.py" ended, send 10 POST requests to the load balancer bound to your auto scaling group. If the server program run on the EC2 instance is non-blocking, now you should have 20 "stress_cpu.py" processes running (t4g nano instance has 2 CPUs, stress_cpu.py automatically starts one stressing process for each CPU, so in total it should be 2*10=20 processes). This will stress your EC2 instance for 10 minutes.

4. Don't stop your EC2 instance, wait for around 5 ~ 9 minutes and check if a new instance is automatically created. If so, it means that your auto scaling group scales up successfully.

5. Make sure this newly created EC2 instance is running your server code automatically. You can test it by sending a GET request to it and see if it returns its private IP address. If so, it means your launch template works.

6. You can now reset your auto scaling group size settings back to: Desired: 1, Minimum: 0, Maximum: 3. Wait for around 10 minutes and see if only 1 instance is kept. When submitting your auto scaling group to the autograder, please always make sure that only 1 instance is running.

Step 2.6. Submission

- In order to grade your work on section 1 and 2, make sure you have:
    - 2 EC2 instances running the server code for section 1

    - A load balancer monitoring the 2 EC2 instances for section 1
    - An auto scaling group for section 2
    - A load balancer monitoring the EC2 instances created by the auto scaling group for section 2 (so basically you should have 1 load balancer for section 1 and 1 load balancer for section 2)
- To submit your code, first fill in the variables in submit.py. Then run submit.py on your local machine. You will receive the autograder's feedback for the current submission, and your grade on Coursera will be updated automatically. Note that the grading process may take 25 minutes. Monitor the submissions section actively for any failure feedback. Intermediate failures will be promptly updated to minimize debugging time.
- Also, **before every submission, make sure updating your submission token,** since the valid time for a token is only 30 minutes. Besides, before submission, **make sure your auto scaling group has only 1 EC2 instance and the desired instance number equals to 1**. After failing the test cases, you might need to reset it.
- In submit.py, make sure you fill in the **DNS address of your load balancer** (not IP address) as well as your school email and personal token.
- Here's a template for your submission script:

```
1
2
3
4
5
6
7
8
9
10
11
```

38

39

40

```
import requests
import json

''' Fill in the following information '''
# General information
YOUR_EMAIL = "" # <put your coursera account email>,
YOUR_SECRET = "" # <put your secret token from coursera>

# <To get full credits, leave this blank. It will run both section 1 and section 2 tests>
SECTION = "1" # <put "1" to test section 1 in isolation; put "2" to test section 2 in isolation>

# Section 1
IP_ADDRESS1 = "" # <put your first EC2 instance's IP address:port> for receiver service
IP_ADDRESS2 = "" # <put your second instance's IP address:port> for receiver service
IP_ADDRESS3 = "" # <put your data access service instance's IP address> DO NOT INCLUD
E PORT HERE
NACL_ID = "" # put NACL ID here
DATA_SERVICE_SUBNET_ID = "" # put subnet ID for data access service here
RECEIVER_APPLICATION_CIDR_RANGE = "" # put CIDR block range for the subnet associat
ed to receiver service instances


# Do not add "https://" to your loadbalancer address.
```

YOUR_LOAD_BALANCER1 = "" # <put your load_balancer address for section 1 (explicitly add :port if port is not 80)>

Section 2

YOUR_LOAD_BALANCER2 = "" # <put your load_balancer address for section 2 (explicitly add :port if port is not 80)>,

''' Don't change the following '''

url = "https://ekwygde36j.execute-api.us-east-1.amazonaws.com/alpha/execution"

```
input = {
        'ip_address1': IP_ADDRESS1,
        'ip_address2': IP_ADDRESS2,
        'ip_address3': IP_ADDRESS3,
        'load_balancer1': YOUR_LOAD_BALANCER1,
        'load_balancer2': YOUR_LOAD_BALANCER2,
        'submitterEmail': YOUR_EMAIL,
        'secret': YOUR_SECRET,
        'section': SECTION,
        'NACL_ID': NACL_ID,
        'data-service-subnet-id': DATA_SERVICE_SUBNET_ID,
        'receiver-application-cidr-range': RECEIVER_APPLICATION_CIDR_RANGE
    }
```

submit

PY File

Note - We post a dummy result on Coursera with all 0s to verify submission token validity. You can ignore this submission as it doesn't reflect the final autograder output.

Step 2.7. shutdown all AWS services

- Remove the auto scaling group or set the desired instance number to 0. Then shutdown your EC2 instances. Please make sure that no new EC2 instance be automatically created by the auto scaling group. Also remove the load balancer and delete your Code Commit repository. Otherwise, Amazon will keep charging you!

Step 2.8. Monitor Auto Grader activity after submission.

Step 1) **Run your submit.py file with a newly generated token.** Once you run your submit.py file using "python3 submit.py". You will see a 200 OK response if the API goes through. For example:

```
200 OK
{"executionArn":"arn:aws:states:us-east-1:913708708374:execution:mp2grader:8da13a53-7ffa-4a63-be0d-3896e27f0c56","startDate":1.705874344468E9}
```

This only means that your file was submitted successfully, and auto grader has started processing your submission. **This response does not mean that your submission passed all test cases.**

Step 2) **Section 1 grader runs (Within the 1st min of submission)**

a) Make sure your instance APIs (Both GET and POST) and Load Balancer are accessible from the internet and respond within a few seconds. Example: Use CURL from a new terminal/cmd. Installing and using curl | Zendesk Developer Docs

- GET: curl http://[publid ip address]:[port]/
- POST: curl -H "content-Type:application/json" -X POST -d "[Json payload]" http://[publid ip address]:[port]/

b) Verify that a GET request to the load balancer returns a new seed number within the 1st min of the auto grader run.

Note: After this step, any known error will be caught and posted on Coursera however, the auto grader will continue to grade Section 2. Any unhandled error will cause the auto grader to crash silently without posting on Coursera. **Verify whether your seed value has changed after 1 min of submitting your solution to avoid silent crashes. Common issues**: a) EC2 instances or the LoadBalancer aren't accessible to public internet. (Make sure your security group allows all inbound traffic) b) LB isn't internet facing. c)Health Checks are failing. d) Get request does not return the seed value as a "string" (Make sure the response isn't of type JSON - Check response headers to be sure) Step 3) **Section 2 Test 1 grader runs - (Within 2 mins of submission)**

Make sure your instance APIs (Both GET and POST) and Load Balancer are accessible from the internet and respond within a few seconds. (Check Step 2 for more)

a) The auto grader will verify the number of EC2 instances active. Make sure only 1 EC2 instance is active during this time.

Note: After this step, any known error will be caught and posted on Coursera and the grader will stop executing further test cases. Any unhandled error will cause the auto grader to crash silently without posting on Coursera.

Common issues: a) Load Balancer isn't accessible to public internet. (Make sure your security group allows all inbound traffic) b) LB isn't internet facing. c) Health Checks are failing.

Step 4) **Section 2 Test 2 grader runs - (Between 2 to 12 mins of submission)**

Make sure your instance APIs (Both GET and POST) and Load Balancer are accessible from the internet and respond within a few seconds. (Check Step 2 for more)

a) Firstly, the auto grader will hit the POST request 10 times on loadbalancer2. Before submitting to auto grader, make sure your POST API is reachable and responds within seconds (runs stress_cpu.py in non-blocking mode using subprocess.Popen()). Also make sure that after hitting 10 consecutive POST requests, the CPU usage goes up to 100% and your auto scaling group adds a new EC2 within 10 mins.

b) After sending 10 POST requests, the auto grader goes to sleep for 10 mins. Monitor your EC2 instance using htop to verify that CPU utilization is at 100% and 10 stress_cpu.py processes are running. **Note:** If your code fails between this time, it's likely that your POST API isn't reachable or is getting timed out.

c) After the 10 min wait, auto grader will verify if you have 2 EC2 instances up. (10 mins of CPU stress has been sufficient for Auto Scaling Group to spin a new EC2 instance. However, if for some reason your ASG takes more than 10 mins given the set-up is correct, reach out to the staff.)

Note: After this step, any known error will be caught and posted on Coursera and the grader will stop executing further test cases. Any unhandled error will cause the auto grader to crash silently without posting on Coursera.

Common issues: a) POST request to Load Balancer is taking too long to respond, causing the auto grader to timeout

Step 5) **Section 2 Test 3 grader runs - (Between 12 to 22 mins of submission)**

a) Similar to Step 4, the auto grader will hit 20 POST requests this time to stress both EC2 instances. Monitor both your EC2 instances for CPU usage and active stress_cpu.py processes.

b) After waiting for 10 more mins, the auto grader will verify whether you have 3 EC2 instances running. Make sure the third instance spins up within 10 mins.

Note: After this step, any known error will be caught and posted on Coursera. Any unhandled error will cause the auto grader to crash silently without posting on Coursera. If everything works correctly, your final score will be posted to Coursera using your email Id and secret token. Your secret token is active for 30 mins and the grading process takes approximately 20 mins. It is therefore important to make sure you use a new token before each submission to see the final results on Coursera.

Common issues: a) POST request to Load Balancer is taking too long to respond, causing the auto grader to timeout

MP2 CONCERNS & FAQs: 1. Load balancer - Failing security checks: ● http://flask.palletsprojects.com/en/1.1.x/quickstart/#quickstart ● $ flask run --host = 0.0.0.0 2. Unable to connect to Ec2 from local machine: ● You don't need to connect to your ec2 using a web browser. You can connect via command line. ● If you want to ping your flask server, you need to include the port number that the flask server is running on. 3. 501 Not Implemented - Failed to connect to load balancer. ● Try deleting the http:// part of your load balancer address in test.py. ● Not supposed to add port number to load balancer url. 4. Load Balancer: EC2 Instances - "OutOfService" ● Ensure to login to EC2 instances via ssh and then run server.py code. 5. Execution of test.py leads to connection timeout: ● Do not run test.py script on the same ec2 instance. Run the test.py program from the local terminal. I.e. Final step has to be done on a separate instance or your local machine. It should not be done on the any of the two ec2 instance. 6. 502 Internal Error and No traffic goes to the ec2 instances ● Looks like the security groups in the load balancer had to be changed to allow all traffic. ● Using application load balancer instead of classic. ● Not able to parse the num value you are sending because it is being received as a byte literal such as this: b'"1000"\n'. ● flask server is returning bytes format of a string instead of just the string itself. 7. Port Mapping FAQ ● Set your load balancer to listen on a particular port (By default 80). If you send a request to the DNS, it'll route the request to port 80 by default. If you're using a different port, specify the port number in the url "DNS:5000" when you submit your MP. Security group should allow 'inbound' traffic on this port. (Let's assume we use port 80 here) ● Once the load balancer receives a request on port 80, it has to forward that request to a target group. (Target groups are just request routers that take requests from load balancer and route them to the individual EC2 instance) ● Your target group

should be set to listen on the port where your EC2 server is running. Let's say we use port 5000. (Outbound rules on your load balancer's security group needs to be in sync with this) ● Now, when you register EC2 instances on your target group, it should route requests to the same port where your EC2 server is running. For example, if your flask is serving on port 5000, your target group should be routing the requests it receives from the load balancer to port 5000 on your EC2 instance. (Again, EC2 security group should allow inbound requests on this port) ● An additional configuration is required for health checks. You can specify a port separately for the target group to perform health checks on. It should match the port number of your server. (5000 in our example)

8. The number of EC2 instances doesn't equal to 2.

● Before every submission, please make sure that there should only be a single instance registered to target group before submission and that instance must be created via launch template. Please deregister any target which is created manually.

● Ensure that the autoscaling capacities (1- desired, 0 - minimum, 3 - maximum) are as instructed before each submission. Also the autoscaling policy is applied which creating autoscaling group as mentioned in the instructions.

How to submit

Copy the token below and run the submission script included in the assignment download. When prompted, use your email address **fridkis2@illinois.edu**.

Generate new token

Your submission token is unique to you and should not be shared with anyone. You may submit as many times as you like.

Like

Dislike

Report an issue